

SEENBIRD AI AGENT · MESSENGER SDK

커스텀 런처 안 읽은 메시지 수 표시 가이드

`FixedMessenger.Launcher + aiAgent.getUnreadMessageCount()`
숫자 뱃지 구현 참고 코드 포함

대상 기능: 런처 미읽음 숫자 뱃지 (커스텀 구현)
기준 버전: @sendbird/ai-agent-messenger-react v1.42.0
문서 발행: 2026-10-02

목차

1. 개요

2. 동작 원리

2.1 조회 필터

2.2 카운트 갱신 시점

3. 참고 코드

3.1 import

3.2 미읽음 카운트 조회 후

3.3 숫자 배지 커스텀 런처

3.4 연결 방법

4. 구현 시 유의 사항

1. 개요

SDK 기본 런처는 안 읽은 메시지가 있을 때 **레드닷(점)**만 표시하며, 숫자 카운트는 내장 기능으로 제공하지 않습니다. 본 문서는 **커스텀 런처를 사용해 숫자 배지를 직접 구현**하는 방법과 바로 사용 가능한 참고 코드를 제공합니다.

구분	기본 런처	커스텀 런처 (본 가이드)
표시 형태	레드닷(점)만	숫자 배지 (1, 2, ... 99+)
갱신 시점	접속·재접속 시점	접속·재접속 + 새 메시지 수신 시 실시간
설정 방법	<code>config.launcher.unreadBadgeEnabled:</code> <code>true</code>	<code>FixedMessenger.Launcher</code> + 직접 조회
구현 주체	SDK	고객사 (SDK는 조회 API와 연동 인터페이스 제공)

커스텀 런처 사용 시 내장 레드닷은 동작하지 않습니다

`FixedMessenger.Launcher` 로 런처를 교체하면 SDK 기본 런처가 렌더링되지 않으므로, `unreadBadgeEnabled` 설정도 적용되지 않습니다. 미읽음 표시는 커스텀 런처가 전적으로 책임지게 되며, 본 가이드의 코드가 그 역할을 수행합니다.

2. 동작 원리

SDK 내부적으로 레드닷을 계산할 때 사용하는 것과 **동일한 조회 API**를 사용하되, 결과를 `boolean( total > 0 )`이 아닌 숫자(`total`)로 그대로 사용하는 방식입니다.

SDK `chatSDK.aiAgent.getUnreadMessageCount(params)`

메신저 컨텍스트에서 `useMessengerContext()` 로 Chat SDK 인스턴스를 얻은 뒤 위 메서드를 호출하면, 다음 형태의 응답을 받습니다.

```
{
  "total": 3,                // 전체 안 읽은 메시지 수 (배지에 표시할 값)
  "aiAgentChannelCount": 3,  // AI 에이전트 채널의 안 읽은 수
  "deskChannelCount": 0,    // Desk(상담사) 채널의 안 읽은 수
  "pinnedChannel": []       // 고정 채널별 안 읽은 수
}
```

왜 `groupChannel.getTotalUnreadMessageCount()` 가 아닌가

Chat SDK의 일반 메서드인 `getTotalUnreadMessageCount()` 는 해당 사용자의 **모든 그룹 채널**을 집계하므로, 동일 앱에서 다른 채팅 기능을 함께 쓰는 경우 무관한 채널까지 포함됩니다. 반면 `aiAgent.getUnreadMessageCount()` 는 AI 에이전트·Desk 채널로 범위가 한정되고 에이전트 ID·대화 상태 필터도 지원하므로 이 방식을 권장합니다.

2.1 조회 필터 (선택)

여러 에이전트를 운영하여 특정 에이전트의 미읽음만 집계해야 하는 경우 파라미터를 전달합니다.

파라미터	타입	필수	설명
<code>aiAgentIds</code>	<code>string[]</code>		집계 대상 에이전트 ID 목록 (기본: 전체)
<code>aiAgentConversationStatusFilter</code>	<code>ConversationStatus[]</code>		대화 상태 필터 (기본: OPEN, CLOSED)
<code>aiAgentChannelFilter</code>	<code>AIAgentChannelFilter</code>		AI 에이전트 채널 포함/제외 (기본: INCLUDE)
<code>deskChannelFilter</code>	<code>DeskChannelFilter</code>		Desk 채널 포함/제외 (기본: INCLUDE)

2.2 카운트 갱신 시점

참고 코드는 다음 4가지 시점에 카운트를 관리합니다. 특히 3번은 SDK 기본 레드닷에는 없는 동작으로, 이를 통해 **실시간 갱신**이 가능해집니다.

시점	처리	목적
최초 진입	연결 상태 확인 후 1회 조회	초기 카운트 표시
연결 / 재연결	<code>ConnectionHandler</code> 로 재조회	백그라운드 복귀·네트워크 복구 대응
새 메시지 수신	<code>GroupChannelHandler</code> 로 재조회	실시간 갱신
메신저 열림	카운트를 0으로 초기화	읽은 것으로 간주, 배지 제거

3. 참고 코드

아래 코드는 샘플 프로젝트의 `web/src/components/UnreadCountLauncher.tsx` 파일로 제공되며, 실제 Dev 테넌트 환경에서 동작을 검증한 코드입니다.

3.1 import

```
import { useCallback, useEffect, useRef, useState } from 'react';
import { useMessengerContext, type FixedLauncherProps } from '@sendbird/ai-agent-messenger-react';
import { ConnectionState, ConnectionHandler } from '@sendbird/chat';
import { GroupChannelHandler } from '@sendbird/chat/groupChannel';
import type { AIAgentGroupChannelUnreadMessageCountParams } from '@sendbird/chat/aiAgent';
```

3.2 미읽음 카운트 조회 후

```
export function useAIAgentUnreadCount(opened: boolean, params?:
AIAgentGroupChannelUnreadMessageCountParams) {
  const { chatSDK } = useMessengerContext();
  const [count, setCount] = useState(0);

  // 비동기 응답이 역순으로 도착할 때 오래된 결과가 최신 값을 덮어쓰지 않도록 요청 순번을 관리
  const requestIdRef = useRef(0);

  const refresh = useCallback(() => {
    const requestId = ++requestIdRef.current;
    chatSDK.aiAgent
      .getUnreadMessageCount(params)
      .then((result) => {
        if (requestId === requestIdRef.current) setCount(result.total);
      })
      .catch(() => {
        // 조회 실패 시 기존 카운트 유지 (뱃지가 깜빡이지 않도록)
      });
  }, [chatSDK, params]);

  // 1) 최초 진입 시 1회 조회
  useEffect(() => {
    if (chatSDK.connectionState === ConnectionState.OPEN) refresh();
  }, [chatSDK, refresh]);

  // 2) 연결/재연결 시 재조회 (백그라운드 복귀, 네트워크 복구 등)
  useEffect(() => {
    const handlerId = 'unread-count-connection';
    chatSDK.addConnectionHandler(
      handlerId,
      new ConnectionHandler({
        onConnected: () => refresh(),
        onReconnectSucceeded: () => refresh(),
      })
    );
    return () => chatSDK.removeConnectionHandler(handlerId);
  }, [chatSDK, refresh]);

  // 3) 실시간 갱신 — 새 메시지 수신 시 즉시 재조회
```

```
// (SDK 기본 레드닷은 이 단계가 없어 접속 시점 기준으로만 갱신됩니다)
useEffect(() => {
  const handlerId = 'unread-count-channel';
  chatSDK.groupChannel.addGroupChannelHandler(
    handlerId,
    new GroupChannelHandler({
      onMessageReceived: () => refresh(),
      onChannelChanged: () => refresh(), // 읽음 처리 등으로 카운트가 줄어드는 경우
    }),
  );
  return () => chatSDK.groupChannel.removeGroupChannelHandler(handlerId);
}, [chatSDK, refresh]);

// 4) 메신저를 열면 읽은 것으로 보고 बै지를 즉시 비움
useEffect(() => {
  if (opened) {
    requestIdRef.current++; // 진행 중인 조회 결과 무시
    setCount(0);
  }
}, [opened]);

return count;
}
```

3.3 숫자 बै지 커스텀 런처

```
export function UnreadCountLauncher({ opened, setOpened, size = 56, style }:
FixedLauncherProps) {
  const unreadCount = useAIAgentUnreadCount(opened);

  return (
    <button
      type="button"
      aria-label={unreadCount > 0 ? `상담 열기, 안 읽은 메시지 ${unreadCount}개` : '상담 열기'}
      onClick={() => setOpened(!opened)}
      style={{
        // SDK가 계산한 배치 스타일(position: fixed, bottom/inset-inline-end, z-index)을 그
        대로 적용합니다.
        // ⚠ position을 relative 등으로 덮어쓰면 런처가 화면 좌측 상단으로 밀려나므로 주의하세요.
        // 아래 बै지는 이 fixed 요소를 기준으로 absolute 배치됩니다.
        ...style,
        width: size,
        height: size,
        padding: 0,
        border: 'none',
        borderRadius: '50%',
        background: '#6210cc',
        cursor: 'pointer',
      }}
    >
    {/* 런처 아이콘 — 자사 이미지()로 교체하셔도 됩니다 */}
    <svg width={size * 0.5} height={size * 0.5} viewBox="0 0 24 24" fill="none"

```

```

stroke="#fff" strokeWidth="2">
  <path d="M21 11.5a8.4 8.4 0 0 1-9 8.4 8.4 8.4 0 0 1-3.8-.9L3 21l1.9-5.7a8.4
8.4 0 0 1-.9-3.8 8.4 8.4 0 0 1 8.4-9 8.4 8.4 0 0 1 8.6 9z" />
</svg>

{/* 숫자 배지 */}
{!opened && unreadCount > 0 && (
  <span
    style={{
      position: 'absolute',
      top: -2,
      insetInlineEnd: -2,
      minWidth: 20,
      height: 20,
      padding: '0 6px',
      borderRadius: 10,
      background: '#e53935',
      color: '#fff',
      fontSize: 12,
      fontWeight: 700,
      lineHeight: '20px',
      textAlign: 'center',
    }}
  >
    {unreadCount > 99 ? '99+' : unreadCount}
  </span>
)}
</button>
);
}

```

3.4 연결 방법

메신저에 커스텀 런처를 전달하면 기본 런처 대신 사용됩니다. 위치·크기는 기존과 동일하게 `FixedMessenger.Style` 로 제어합니다.

```

<FixedMessenger
  appId={params.appId}
  aiAgentId={params.aiAgentId}
  userSessionInfo={sessionInfo}
  state={{ opened, setOpened }}
>
  <FixedMessenger.Launcher component={UnreadCountLauncher} />
  <FixedMessenger.Style position="end-bottom" launcherSize={56} margin={{ end: 24,
bottom: 28 }} />
</FixedMessenger>

```

4. 구현 시 유의 사항

1) SDK 배치 스타일을 덮어쓰지 마세요

SDK는 커스텀 런처에 `style` 을 주입하며, 여기에는 `position: fixed`, `bottom`, `inset-inline-end`, `z-index` 가 포함되어 있습니다. 배지 배치를 위해 `position: relative` 등으로 덮어쓰면 **런처가 화면 좌측 상단으로 밀려나므로** 주의가 필요합니다. 주입된 `position: fixed` 자체가 배지(absolute)의 기준점이 되므로 그대로 두면 됩니다.

2) 핸들러는 반드시 해제하세요

`ConnectionHandler` 와 `GroupChannelHandler` 는 고유 key로 등록되므로, `useEffect` 의 `cleanup` 에서 `removeConnectionHandler` / `removeGroupChannelHandler` 를 호출해 중복 등록과 메모리 누수를 방지해야 합니다.

3) 응답 순서 역전 처리

짧은 시간에 조회가 여러 번 발생하면 응답이 역순으로 도착해 오래된 값이 최신 값을 덮어쓸 수 있습니다. 참고 코드는 요청 순번(`requestIdRef`)을 비교해 최신 요청의 결과만 반영합니다.

4) 접근성

숫자만 시각적으로 표시하면 스크린리더 사용자가 인지할 수 없으므로, 참고 코드와 같이 `aria-label` 에 미완성 개수를 함께 서술해 주세요.

5) 표시 한계값

카운트가 큰 경우 배지가 런처를 가릴 수 있으므로 `99+` 형태의 상한 처리를 권장합니다.

메신저를 로드하지 않으면 카운트 조회도 불가합니다

본 방식은 메신저(SDK)가 로드되어 Sendbird에 연결된 상태를 전제로 합니다. 사용자가 해당 화면에 진입하지 않은 상태에서의 알림이 필요하다면 앱 푸시 알림으로 보완해야 합니다.

※ 본 가이드는 Sendbird 내부 검증 결과를 토대로 작성되었습니다. 사양 변경이 있을 수 있으므로 운영 적용 전 담당 CSM 또는 SE에게 최신 상태를 확인해 주시기 바랍니다.